

SYSTEM DESIGN & ARCHITECTURE

pySAR App

Protein Sequence–Activity Relationship modelling — full-stack web application

Frontend · React 18 SPA on Vercel (static + CDN)

Backend · FastAPI on Google Cloud Run (europe-west1)

Science core · pySAR + scikit-learn + protpy + aaindex

Generated 5 July 2026 · reflects `main` branch

1 · Overview

pySAR is a web application for **protein Sequence–Activity Relationship (SAR) modelling**. A user uploads a labelled dataset of protein sequences, configures an encoding strategy and machine-learning model, submits a job, and receives ranked results with performance metrics (R^2 , RMSE, Pearson) and interactive charts.

The system is split across two hosting platforms connected over HTTPS REST:

- **A static React SPA** served from Vercel's CDN — all UI, client-side dataset parsing, and static fallbacks for reference data.
- **A stateful FastAPI backend** on Cloud Run — runs the CPU-heavy encoding/training in background threads and exposes a polled job API.

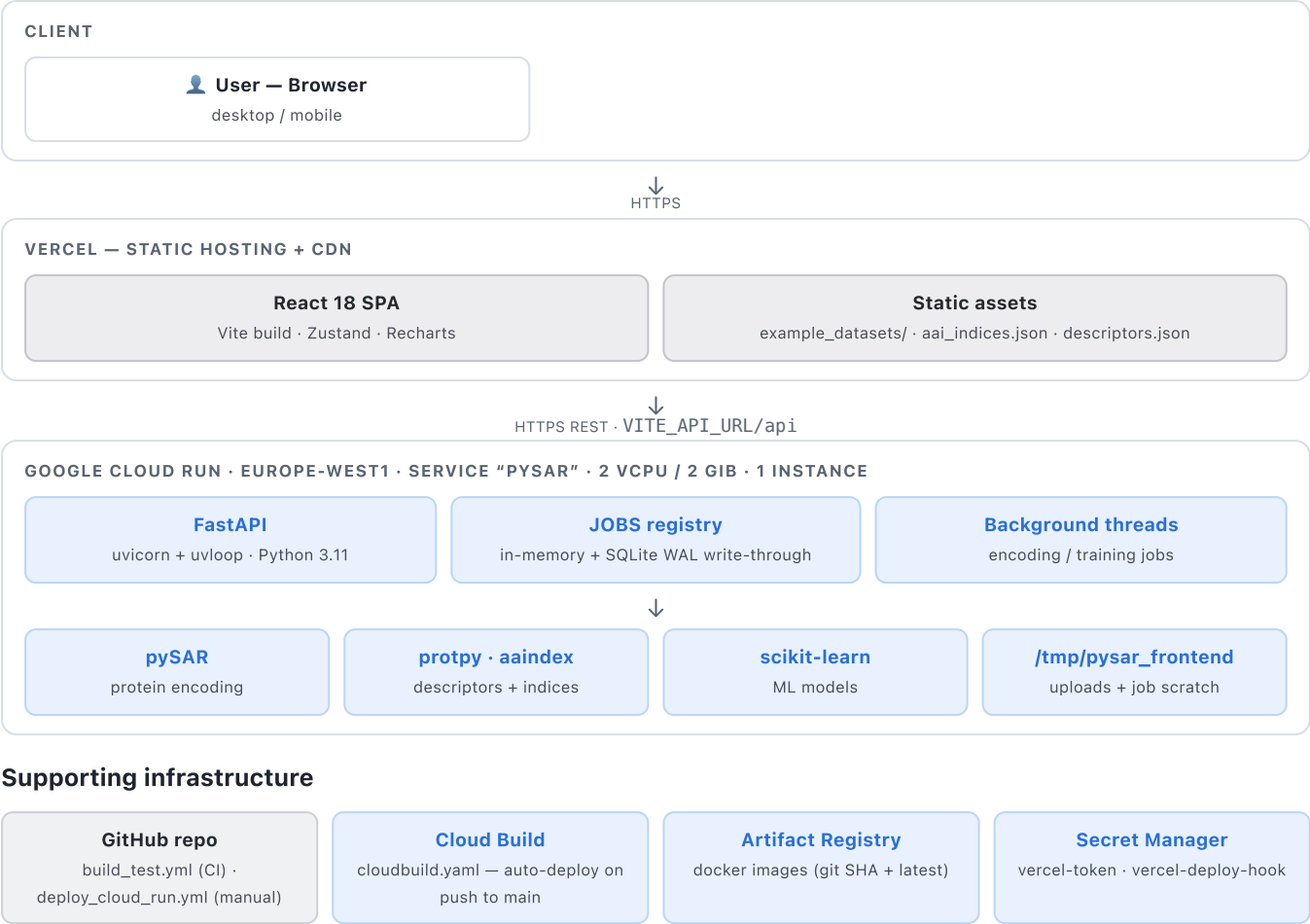
The split keeps the interactive UI globally fast and cheap while isolating the heavy Python scientific stack (numpy/scipy/scikit-learn/pySAR) in a single always-warm compute instance.

Core design decisions

Decision	Rationale
Fire-and-poll job model	Encoding can take minutes; a synchronous request would hit HTTP timeouts. <code>/encode</code> returns a <code>job_id</code> immediately and the client polls.
Single instance (<code>max-instances=1</code>)	Job state lives in-process; multiple instances would fragment it. CPU is kept always-allocated so background threads keep running between polls.
SQLite write-through	The in-memory job registry is mirrored to SQLite (WAL) so completed jobs survive an instance restart / redeploy.
Client-side example parsing	Built-in datasets are parsed in the browser; the file is only uploaded lazily when a job is actually submitted.
Static fallbacks	Reference data (AAI indices, descriptors) ships as bundled JSON, so the explorers work even when the backend is cold or offline.

2 · System topology

☐ Vercel ☐ Google Cloud ☐ GitHub / CI



On deploy, CI/Cloud Build pushes the image, runs `gcloud run deploy`, then calls the Vercel API to sync `VITE_API_URL` to the new Cloud Run URL and triggers a frontend redeploy.

3 · Request & data flow

The end-to-end journey from landing to results:

- 1 **Wake / health probe.** On mount the SPA calls `GET /api/health`. If the backend is cold (free-tier spin-down), it polls with backoff for up to 60 s and shows a “waking up the server...” banner.
- 2 **Upload dataset.** `POST /api/upload` (multipart, CSV/TSV/TXT). Backend validates, stores under `/tmp`, returns `{ file_id, columns, preview, stats }`. Built-in example datasets are parsed client-side and uploaded only on job submit.
- 3 **Data quality (optional).** Deduplicate, fix missing sequences/activity, or winsorize outliers via `/api/dataset/{id}/...` endpoints, each returning a fresh `file_id`.
- 4 **Configure.** User picks an encoding strategy (AAI / descriptor / combined / embedding), descriptors, DSP settings, ML algorithm, and CV/holdout split. A live JSON config preview is editable and exportable.
- 5 **Submit job.** `POST /api/encode` validates + rate-limits, assigns a UUID, records a pending job, spawns a **daemon background thread**, and returns `{ job_id }` instantly.
- 6 **Encode & train.** The thread runs pySAR: numerically encodes sequences, optionally applies DSP (FFT spectrum), trains + evaluates every model combination, writing progress and top-10 partial results back to the job record.
- 7 **Poll.** The SPA polls `GET /api/jobs/{job_id}` (~2 s, exponential backoff when backgrounded) reading `status · progress · log · partial_results`.
- 8 **Results.** On completion the full ranked DataFrame + best-model metadata are returned. UI renders a sortable table and Recharts scatter/bar/line plots.
- 9 **Act on results.** Export PDF/Excel/repro-bundle, download the pickled best model, **predict** new sequences (`/predict`), or create a read-only **share link** (`/share`).

Why polling, not websockets: jobs are decoupled from any single request, so a dropped connection, a refresh, or navigating away never kills the run — the background thread keeps going and the client re-attaches on the next poll.

4 · Frontend architecture

Deployed: Vercel static SPA + CDN · URL: `https://pysar-app.vercel.app` · Build: Vite 5

Stack

Concern	Library
UI framework	React 18.3
Build	Vite 5 + plugin-react
Styling	Tailwind CSS 3.4 + PostCSS
State	Zustand 4.5 (+ persist)
HTTP	Axios 1.7
Charts	Recharts 2.15
Icons	@heroicons/react 2.2
File input	react-dropzone 14.3
Toasts	react-hot-toast 2.4
Export	jsPDF 4.2 · xlsx (SheetJS) 0.18
Tests	Vitest + Testing Library · Playwright (E2E)

State management (Zustand)

A single flat store (`appStore.js`). Only a selective subset persists to `localStorage` :

Persisted	Purpose
<code>darkMode</code>	Theme preference
<code>showLanding</code>	Show landing on next visit
<code>config</code>	Last-used config
<code>aaiIndicesCache</code>	Cached index list
<code>jobHistory</code>	Past job metadata

Transient state — dataset, current step, active job, results — is intentionally **not** persisted, so a refresh returns a clean slate.

Component structure

```
App.jsx      ← root · step router · backend wake probe · "waking up" banner
├─ LandingPage.jsx      ← intro / entry screen
├─ Layout.jsx           ← nav, dark-mode, explorer launchers, background job polling
├─ ErrorBoundary.jsx    ← React error boundary
├─ steps/
│   ├─ Step1Upload.jsx  ← upload (drag-drop / picker / examples)
│   │   └─ DatasetPreview.jsx ← table preview + data-quality tools
│   ├─ Step2Configure.jsx ← three-panel config editor
│   │   ├─ ModelConfig.jsx  ← algorithm, CV/holdout, hyperparams
│   │   ├─ DescriptorConfig.jsx ← descriptor selection + params
│   │   └─ DSPConfig.jsx    ← FFT / window / filter settings
│   │       └─ ConfigPreview.jsx ← live JSON preview, import/export
│   ├─ Step3Encode.jsx   ← submit, progress polling, queue, offline banner
│   └─ Step4Results.jsx  ← results table, sort, export, predict
│       ├─ ResultsCharts.jsx ← scatter / bar / line
│       └─ ModelInsights.jsx ← feature importance, CV folds
├─ components/
│   ├─ JobsPanel.jsx · SharedResults.jsx · ModelExplorer.jsx
│   └─ AaiExplorer.jsx · DescriptorExplorer.jsx
└─ HelpTooltip.jsx · HowToModal.jsx · Skeleton.jsx · reproExport
```

HTTP client (`utils/api.js`)

- Single Axios instance; `baseUrl = VITE_API_URL/api` in prod, or `/api` via the Vite dev proxy locally.
- 30 s default timeout; upload/encode set `timeout: 0` for large files.
- **Retry interceptor** — up to 3 retries with exponential backoff (1→2→4 s) on network errors / 502 / 503, for **GET and POST** (cold-start signals mean the request never reached the app, so replaying upload/encode is safe).
- **Cold-start wake** — `wakeBackend()` polls `/health` up to ~60 s to warm a spun-down backend before the first real call, driving the “waking up...” banner.
- **Static fallback** — `getAaiIndicesFull()` / `getDescriptors()` fall back to bundled CDN JSON when the backend is unreachable.

Dev proxy

In development Vite proxies all `/api` requests to `http://localhost:8000` , so the frontend talks to a local FastAPI instance with no CORS setup. In production the Cloud Run URL is baked in at build time via `VITE_API_URL` .

5 · Backend architecture

Deployed: Google Cloud Run · `europa-west1` · service `pysar` · **Server:** uvicorn + uvloop, single worker · **Language:** Python 3.11

Stack

Concern	Library	Concern	Library
Web framework	FastAPI ≥0.115	Protein encoding	pySAR (GitHub, pinned SHA)
ASGI server	uvicorn + uvloop	Descriptors	protpy ≥1.3
Data	pandas ≥2.0	AAI indices	aaindex ≥1.2
Numerical	numpy · scipy	Validation	Pydantic v2
ML	scikit-learn	Persistence	SQLite (stdlib, WAL)
Embeddings (opt.)	torch · transformers	Tests	pytest · httpx

pySAR is installed from GitHub pinned to a **commit SHA** (not `@master`) so upstream pushes can't silently change production behaviour — bumps are deliberate. The `pysar` PyPI name is an unrelated JPL InSAR tool.

API routes

Method	Path	Purpose
GET	/api/health	Liveness probe → <code>{"status":"ok"}</code>
GET	/api/version	Backend / pySAR / Python versions
GET	/api/aa1-indices · /aa1-indices-full	566 AAI1 codes; codes+titles for the explorer
GET	/api/descriptors	Descriptor catalogue with metadata
GET	/api/embeddings/status	Whether PLM-embedding strategy is available
POST	/api/upload · /upload-descriptors	Upload dataset / pre-computed descriptor CSV
GET	/api/dataset/{id}/rows	All rows of an uploaded dataset
POST	/api/dataset/{id}/deduplicate	Remove duplicate sequences
POST	/api/dataset/{id}/fix-missing-sequences	Drop rows with null sequences
POST	/api/dataset/{id}/fix-missing-activity	Fill / remove missing activity values
POST	/api/dataset/{id}/fix-outliers	Winsorize / remove outlier activities
GET	/api/example-datasets	List built-in sample datasets
POST	/api/example-dataset/{name}	Load a built-in dataset by name
POST	/api/encode	Submit encoding job → <code>{ job_id }</code>
GET	/api/jobs · /jobs/{id}	List jobs; poll one job's status + results
POST	/api/jobs/{id}/cancel	Cancel a running job
DELETE	/api/jobs/{id}	Remove a completed job from the registry
GET	/api/jobs/{id}/model	Download the pickled best model
POST	/api/jobs/{id}/predict	Score new sequences with the best model
POST	/api/jobs/{id}/share	Mint a read-only share token
GET	/api/share/{token}	Fetch shared results (no session needed)

Interactive Swagger UI is auto-generated at `/api/docs` .

Job lifecycle

- 1 **Admit.** `/encode` validates the request, enforces per-IP concurrency (max 3) and a model-count ceiling inside one lock (avoids TOCTOU), then records a `pending` job and persists it to SQLite.
- 2 **Run.** A daemon thread encodes + trains, updating `progress · models_completed · partial_results` on the job record; a cancel `threading.Event` allows cooperative interruption.
- 3 **Complete.** Full results, best-model pickle, feature importance and CV scores are written back and mirrored to SQLite.
- 4 **Evict.** An hourly sweep removes temp files >6 h old, evicts completed jobs past a 30-min TTL, prunes ghost/orphan records, and clears expired rate-limit buckets.

Duration vs platform timeout: because encoding runs in a background thread (not in the HTTP request), the 30-min `MAX_JOB_DURATION_SECS` cap is *not* bound by Cloud Run's request timeout (3600 s). The load-bearing platform requirement is CPU staying allocated between polls — provided by `--no-cpu-throttling` + `--min-instances=1`.

Rate limiting

Custom sliding-window limiter (no dependency) as ASGI middleware:

Endpoint	Limit / IP
POST <code>/encode</code>	5 / 60 s
POST <code>/upload</code>	20 / 60 s

Client IP is read from `X-Forwarded-For` counting `TRUST_PROXY_HOPS` from the right (forgery-resistant). Unset ⇒ socket peer only.

CORS & sessions

Allowed origins built at startup: `localhost:5173`, `https://{VERCEL_URL}`, and `CORS_ORIGIN`. Ownership of jobs/datasets is scoped by an unguessable per-browser `X-Session-Id` token (minted server-side, echoed back), so one client can't see another's jobs even behind shared NAT.

Startup / maintenance threads

Thread	Purpose
<code>_prewarm_pysar</code>	Imports <code>pySAR.Encoding</code> at boot so the first job avoids the cold import cost
<code>_cleanup_upload_dir</code>	Hourly sweep: temp files, job TTL eviction, ghost pruning, rate-limit bucket cleanup
<code>_load_jobs_from_db</code>	Repopulates the in-memory registry from SQLite on startup (restart durability)

Logs are emitted as single-line JSON so GCP Cloud Logging indexes them by severity natively.

6 · Infrastructure & deployment

Cloud Run service

Service	pysar
Region	europe-west1
Project	pysar-493713
Memory / CPU	2 GiB · 2 vCPU
CPU throttling	off (always allocated)
Min / max instances	1 / 1
Request timeout	3600 s
Startup CPU boost	enabled
Auth	public (allUsers → run.invoker)

Vercel project

Framework	None (static)
Build	cd frontend && npm run build
Output	frontend/dist
Prod URL	pysar-app.vercel.app
Key env	VITE_API_URL → Cloud Run URL

Registries & secrets

Images → Artifact Registry (git SHA + `latest`). `vercel-token` & `vercel-deploy-hook` in Secret Manager, injected into Cloud Build only.

Docker image — two-stage build

```
Stage 1 builder (python:3.11-slim)
apt: gcc g++ git                                ← native build tools + pip VCS installs
pip install --prefix=/install -r requirements.txt

Stage 2 runtime (python:3.11-slim)
COPY --from=builder /install /usr/local          ← compiled packages only, no build tools
COPY backend/ · example_datasets/
USER appuser (non-root) · WORKDIR /app/backend · EXPOSE 8080
CMD uvicorn main:app --host 0.0.0.0 --port ${PORT:-8080} --workers 1 --loop uvloop
```

CI/CD pipelines

Pipeline	Trigger
build_test.yml GitHub Actions CI	push / PR — vitest, flake8 + pytest, Playwright E2E
cloudbuild.yaml Cloud Build	auto on push to <code>main</code> — build, deploy, sync Vercel env, redeploy hook
deploy_cloud_run.yml GitHub Actions	manual <code>workflow_dispatch</code> — same deploy steps, hand-triggered

Deploy sequence (both pipelines):

- 1 docker build → push to Artifact Registry
- 2 `gcloud run deploy` the new image
- 3 PATCH `VITE_API_URL` in Vercel to the Cloud Run URL
- 4 Trigger a Vercel production redeploy (deploy hook)

7 · pySAR scientific core

pySAR is the scientific engine. Its `Encoding` class takes a sequence dataset, an encoding strategy, and an ML model, and returns a ranked results DataFrame (R^2 , RMSE, MSE, Pearson per model combination).

Encoding strategies

Strategy	Description
<code>aai</code>	Encode sequences with AAI1 amino-acid indices (physico-chemical scales). Each of 566 indices is tried; optional DSP (FFT) is applied; an ML model is trained & evaluated.
<code>descriptor</code>	Encode with protein descriptors from protpy — compositional, structural and autocorrelation features derived from the sequence.
<code>aai_descriptor</code>	Concatenate AAI-encoded features with descriptor features before training.
<code>embedding</code>	Optional PLM embeddings (ESM-2 / ProtT5) — gated on <code>torch</code> + <code>transformers</code> being installed on the backend.

Descriptors (protpy)

AA / dipeptide / tripeptide composition · Moran, Geary & Moreau-Broto autocorrelation · CTD (composition / transition / distribution) · sequence-order coupling · quasi-sequence-order · pseudo-AAC & amphiphilic pseudo-AAC · k-mer & reduced-alphabet composition.

ML models (scikit-learn)

PLS regression · linear / ridge / lasso / elastic-net · random forest · gradient boosting · SVR · k-NN · decision tree · AdaBoost · bagging. Regression is the primary task; classification is also supported.

Digital signal processing (pyDSP)

When `use_dsp: true`, AAI-encoded sequences pass through a signal-processing pipeline — FFT power/magnitude/real/imaginary spectrum, optional window functions, optional Savitzky-Golay / median filter — and the ML model is trained on the resulting frequency-domain features.

Local development

```
# Backend (terminal 1)           # Frontend (terminal 2)
pip install -r backend/requirements.txt  npm --prefix frontend install
python3 -m uvicorn backend.main:app \    npm --prefix frontend run dev
--reload --port 8000                  # → http://localhost:5173

# Vite proxies /api → localhost:8000, so no CORS setup is needed locally.
```